

Міністерство освіти та науки України

Національний університет «Львівська політехніка»

Інститут телекомунікацій, радіоелектроніки та електронної техніки



ЗВІТ

З дисципліни

“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
Студент групи ІХ-22
Брона М.В.

Прийняв:
Гордійчук-Бублівська О.В

Львів 2026

Тема: Асинхронне програмування в телекомунікаційних системах: взаємодія з базою даних через SQLAlchemy.

Мета: Ознайомитися з принципами асинхронного програмування в Python за допомогою модуля `asyncio`, навчитися працювати з асинхронними запитами до бази даних через SQLAlchemy, реалізувати систему збору та обробки даних про мережеві вузли.

Завдання

1. Створити базу даних та таблицю `nodes`, яка зберігатиме інформацію про мережеві вузли (`ID`, `IP-адресу`, `статус`).
2. Реалізувати асинхронну функцію для отримання списку вузлів із бази.
3. Реалізувати асинхронну систему збору статусів вузлів (імітація мережевих запитів).
4. Зберігати отримані дані у базі за допомогою SQLAlchemy.
5. Продемонструвати та пояснити результати виконання програми (створення таблиці, додавання 10-15 вузлів, моніторинг з оновленням статусу, виведення списку вузлів до та після оновлення і т.д.).

--- СТАН ДО ОНОВЛЕННЯ ---

ID	IP Address	Status	Last Seen
1	192.168.1.1	unknown	Never
2	192.168.1.2	unknown	Never
3	192.168.1.3	unknown	Never
4	192.168.1.4	unknown	Never
5	192.168.1.5	unknown	Never
6	192.168.1.6	unknown	Never
7	192.168.1.7	unknown	Never
8	192.168.1.8	unknown	Never
9	192.168.1.9	unknown	Never
10	192.168.1.10	unknown	Never
11	192.168.1.11	unknown	Never
12	192.168.1.12	unknown	Never
13	192.168.1.13	unknown	Never
14	192.168.1.14	unknown	Never
15	192.168.1.15	unknown	Never

[ПОШУК] Починаємо асинхронне опитування вузлів...

[...] Перевірка вузла 192.168.1.1...
[...] Перевірка вузла 192.168.1.2...
[...] Перевірка вузла 192.168.1.3...
[...] Перевірка вузла 192.168.1.4...
[...] Перевірка вузла 192.168.1.5...
[...] Перевірка вузла 192.168.1.6...
[...] Перевірка вузла 192.168.1.7...
[...] Перевірка вузла 192.168.1.8...
[...] Перевірка вузла 192.168.1.9...
[...] Перевірка вузла 192.168.1.10...
[...] Перевірка вузла 192.168.1.11...
[...] Перевірка вузла 192.168.1.12...
[...] Перевірка вузла 192.168.1.13...
[...] Перевірка вузла 192.168.1.14...
[...] Перевірка вузла 192.168.1.15...
[OK] Вузол 192.168.1.1 статус: online
[OK] Вузол 192.168.1.5 статус: online
[OK] Вузол 192.168.1.13 статус: online
[OK] Вузол 192.168.1.14 статус: online
[OK] Вузол 192.168.1.6 статус: offline
[OK] Вузол 192.168.1.9 статус: online
[OK] Вузол 192.168.1.2 статус: online
[OK] Вузол 192.168.1.11 статус: offline
[OK] Вузол 192.168.1.8 статус: online
[OK] Вузол 192.168.1.7 статус: online
[OK] Вузол 192.168.1.3 статус: online
[OK] Вузол 192.168.1.10 статус: online
[OK] Вузол 192.168.1.15 статус: offline
[OK] Вузол 192.168.1.12 статус: offline
[OK] Вузол 192.168.1.4 статус: online

[УСПІХ] Всі статуси в базі даних оновлено.

[OK] Вузол 192.168.1.4 статус: online

[УСПІХ] Всі статуси в базі даних оновлено.

--- СТАН ПІСЛЯ МОНІТОРИНГУ ---

ID	IP Address	Status	Last Seen
1	192.168.1.1	online	01:26:24
2	192.168.1.2	online	01:26:24
3	192.168.1.3	online	01:26:24
4	192.168.1.4	online	01:26:24
5	192.168.1.5	online	01:26:24
6	192.168.1.6	offline	01:26:24
7	192.168.1.7	online	01:26:24
8	192.168.1.8	online	01:26:24
9	192.168.1.9	online	01:26:24
10	192.168.1.10	online	01:26:24
11	192.168.1.11	offline	01:26:24
12	192.168.1.12	offline	01:26:24
13	192.168.1.13	online	01:26:24
14	192.168.1.14	online	01:26:24
15	192.168.1.15	offline	01:26:24

[fetch_nodes] Виведення вузлів через окрему асупс-функцію:

ID	IP Address	Status
1	192.168.1.1	online
2	192.168.1.2	online
3	192.168.1.3	online
4	192.168.1.4	online
5	192.168.1.5	online
6	192.168.1.6	offline
7	192.168.1.7	online
8	192.168.1.8	online
9	192.168.1.9	online
10	192.168.1.10	online
11	192.168.1.11	offline
12	192.168.1.12	offline
13	192.168.1.13	online
14	192.168.1.14	online
15	192.168.1.15	offline

PS C:\Users\Marko>

```

1 import asyncio
2 import aiosqlite
3 import random
4 from datetime import datetime
5
6 DB_NAME = 'network_monitor.db'
7
8
9 async def init_db():
10     async with aiosqlite.connect(DB_NAME) as db:
11         await db.execute("DROP TABLE IF EXISTS nodes")
12         await db.execute('''
13             CREATE TABLE nodes (
14                 id INTEGER PRIMARY KEY AUTOINCREMENT,
15                 ip_address TEXT,
16                 status TEXT,
17                 last_seen TIMESTAMP
18             )
19         ''')
20
21     nodes = [(f"192.168.1.{i}", "unknown", None) for i in range(1, 16)]
22     await db.executemany(
23         "INSERT INTO nodes (ip_address, status, last_seen) VALUES (?, ?, ?)",
24         nodes
25     )
26     await db.commit()
27     print("[!] Базу створено. Додано 15 вузлів.\n")
28
29
30 async def ping_node(ip):
31     """Асинхронна імітація перевірки вузла (ping)."""
32     print(f"[...] Перевірка вузла {ip}...")
33     await asyncio.sleep(random.uniform(0.3, 1.5))
34     status = random.choice(["online", "online", "online", "offline"])
35     print(f"[OK] Вузол {ip} статус: {status}")
36     return ip, status
37
38
39 async def display_nodes(label):
40     """Виведення поточного списку вузлів із бази даних."""
41     print(f"\n--- {label} ---")
42     print(f"{'ID':<4} | {'IP Address':<15} | {'Status':<12} | {'Last Seen'}")
43     print("-" * 35)
44     async with aiosqlite.connect(DB_NAME) as db:
45         async with db.execute("SELECT * FROM nodes") as cursor:
46             for row in cursor:
47                 last_seen = row[3] if row[3] else "Never"
48                 print(f"{row[0]:<4} | {row[1]:<15} | {row[2]:<12} | {last_seen}")
49
50
51 async def fetch_nodes():
52     """Асинхронна функція для отримання списку вузлів із бази."""
53     async with aiosqlite.connect(DB_NAME) as db:
54         db.row_factory = aiosqlite.Row
55         async with db.execute("SELECT id, ip_address, status FROM nodes") as cursor:
56             rows = await cursor.fetchall()
57
58     print(f"\n{'ID':<5} | {'IP Address':<15} | {'Status':<12}")
59     print("-" * 35)
60     for row in rows:
61         print(f"{row['id']:<5} | {row['ip_address']:<15} | {row['status']:<12}")
62
63
64 async def run_monitoring():
65     """Головна функція: ініціалізація, моніторинг, оновлення та виведення."""
66     # 1. Ініціалізація БД
67     await init_db()
68
69     # 2. Стан ДО оновлення
70     await display_nodes("СТАН ДО ОНОВЛЕННЯ")
71
72     # 3. Асинхронне опитування всіх вузлів
73     print(f"\n[ПОШУК] Починаємо асинхронне опитування вузлів...\n")
74     async with aiosqlite.connect(DB_NAME) as db:
75         async with db.execute("SELECT ip_address FROM nodes") as cursor:
76             ips = [row[0] for row in await cursor.fetchall()]
77
78     results = await asyncio.gather(*(ping_node(ip) for ip in ips))
79
80     # 4. Оновлення статусів у БД
81     now = datetime.now().strftime("%H:%M:%S")
82     update_data = [(status, now, ip) for ip, status in results]
83     await db.executemany(
84         "UPDATE nodes SET status = ?, last_seen = ? WHERE ip_address = ?",
85         update_data
86     )
87     await db.commit()
88     print("\n[УСПІХ] Всі статуси в базі даних оновлено.")
89
90     # 5. Стан ПІСЛЯ моніторингу
91     await display_nodes("СТАН ПІСЛЯ МОНІТОРИНГУ")
92
93     # 6. Додаткове виведення через fetch_nodes
94     print("\n[fetch_nodes] Виведення вузлів через окрему async-функцію:")
95     await fetch_nodes()
96
97
98 if __name__ == "__main__":
99     try:
100         asyncio.run(run_monitoring())
101     except KeyboardInterrupt:
102         pass
103

```

Висновок:

Під час виконання лабораторної роботи я ознайомився з принципами асинхронного програмування в середовищі Python, використовуючи модуль `asyncio`. Практична частина була зосереджена на реалізації асинхронної взаємодії з базами даних шляхом впровадження ORM-інструментарію `SQLAlchemy`